Unit_3. Functions, Scoping and Abstraction

What is Function?

If a group of statements is repeatedly required then it is not recommended to write these statements everytime seperately.

We have to define these statements as a single unit and we can call that unit any number of times based on our requirement without rewriting.

This unit is nothing but function.
The main advantage of functions is code Reusability.

Types Of Function:

Python supports 2 types of functions.

(1) Built in Functions.

(2) User Defined Functions.

(1) Built in Functions: The functions which are coming along with Python software automatically, are called built in functions or pre defined functions.

Example:
type() , input()
(2) User Defined Functions: The functions which are developed by programmer explicitly according to business requirements, are called user defined functions.
Syntax to Create (Declaring) User defined Functions:

```
def function_name(parameters) :
        body of function
        return value
```

While creating functions we can use 2 keywords.
(1) def (mandatory) (2) return (optional)

Parameters:
Parameters are inputs to the function.If a function contains parameters, then at the time of calling, compulsory we should provide values otherwise, we will get error.

In [1]:
```
#Write a function to take name of the student as
#input and print wish message by name.
def wish(name):
    print("Hello",name," Good Morning")
wish("ABC")    # Invoking or Calling a function
```

```
Hello ABC  Good Morning
```

In [2]:
```
wish("Ram")
```

```
Hello Ram  Good Morning
```

Return Statement:

Function can take input values as parameters and executes logic, and returns output to the caller with return statement.

In [3]:
```
#Write a Function to accept 2 Numbers as Input and return Sum.
def add(x,y):
    return x+y
result=add(20,10)  # Invoking add function and sum will
                        #be stored in variable result.

print("The sum is",result)
print("The sum is",add(200,100))
```

```
The sum is 30
The sum is 300
```

If we are not writing return statement then default return value is None.

In [4]:
```
def add(x,y):
    x+y
result=add(20,10)
print("The sum is",result)
print("The sum is",add(200,100))
```

```
The sum is None
The sum is None
```

Different categories of user defined functions

In [5]:
```
# function with no parameter and no return type.
def printline():
    s=input("Enter name ")
    print(s)
printline() #calling function
```

```
Enter name SHP
SHP
```

In [6]:
```
# function with parameter and no return type
def printline(s):
    print(s)
s=input("Enter name ")
printline(s) #calling function
```

```
Enter name SHP
SHP
```

In [7]:
```
# function with no parameter and with return type
def printline():
    s=input("Enter name ")
    return s
t=printline() #calling function
print(t)
```

```
Enter name SHP
SHP
```

In [10]:
```
# function with parameter and with return type
def printline(s):
    return s
s=input("Enter name ")
t=printline(s)
print(t)
```

```
Enter name SHP
SHP
```

In [11]:
```
#returning multiple values from function
def sum_sub(a,b):
    sum=a+b
    sub=a-b
    return sum,sub
x,y=sum_sub(50,20)
print("addition=",x)
print("subtraction=",y)
```

```
addition= 70
subtraction= 30
```

In [12]:
```
#returning multiple values from function
def calc(a,b):
    sum=a+b
    sub=a-b
    mul=a*b
    div=a/b
    return sum,sub,mul,div
t=calc(50,10)
print("Results are: ")
for i in t:
    print(i)
```

```
Results are:
60
40
500
5.0
```

Function Specification:

One of the the most important aspects of writing a quality Python function is proper specification.

A specification actually has a very precise definition and implementation for a Python function.

In practice, a specification include a debugging and a docstring.

Debugging : It is the process of finding out why a program does not work, and then fixing it.

Docstring : The text between the triple quotation marks is called a docstring in Python.

```
By convention,
Python programmers use docstrings to provide specifications
of functions.
These docstrings can be accessed using the built-in function help.
```

Python programmers use docstrings to provide specifications of functions.

Discorting can be one liner and multiliner

1. One Liner

In [13]:
```
#one line doc string
def square(x):
 """ Argument give square of x"""
 return x*x
print(square.__doc__)
```

```
 Argument give square of x
```

2. Multi liner

In [25]:
```
def sum_sub(a,b):
 """
 returns the sum of two integers
 returns difference of two integers
 """
 sum=a+b
 sub=a-b
 return sum,sub
x,y=sum_sub(50,20)
print(sum_sub.__doc__)
```

```
 returns the sum of two integers
 returns difference of two integers
```

Parameters and Arguments:

Types Of Arguments:

```
def  f1(a,b):
 ------
------
f1(10,20)
```

a, b are formal arguments where as 10,20 are actual arguments.

There are 4 types of actual arguments which are allowed in Python.

Positional Arguments

Keyword Arguments

Default Arguments

Variable Length Arguments

Positional Arguments:

These are the arguments passed to function in correct positional order.

The number of arguments and position of arguments must be matched.

If we change the order then result may be changed.

If we change the number of arguments then we will get error.

In [29]:
```
def sub(a, b):
    print(a-b)
sub(100, 200)
sub(200, 100)
```

```
-100
100
```

Keyword Arguments:

We can pass argument values by keyword i.e by parameter name.

In [31]:
```
def wish(name,msg):
    print("Hello",name,msg)
wish(name="Arman",msg="Good Morning")
wish(msg="Good Morning",name="Arman")
```

```
Hello Arman Good Morning
Hello Arman Good Morning
```

Here the order of arguments is not important but number of arguments must be matched.

We can use both positional and keyword arguments simultaneously.

But first we have to take positional arguments and

then keyword arguments, otherwise we will get syntax error.

In [33]:
```
def wish(name,msg):
    print("Hello",name,msg)
wish("Arman","Good Morning")
```

```
Hello Arman Good Morning
```

In [34]:
```
wish("Arman",msg="Good Morning")
```

```
Hello Arman Good Morning
```

In [35]:
```
wish(name="Arman","Good Morning")
```

```
  File "<ipython-input-35-c7bad25365fe>", line 1
    wish(name="Arman","Good Morning")
                      ^
SyntaxError: positional argument follows keyword argument
```

Default Arguments:

Sometimes we can provide default values for our positional arguments.

In [37]:
```
def wish(name="Guest"):
    print("Hello",name,"Good Morning")
wish("Arman")
```

```
Hello Arman Good Morning
```

In [38]:
```
wish()
```

```
Hello Guest Good Morning
```

If we are not passing any name then only default value will be considered.

After default arguments we should not take non default arguments.

```
def wish(name="Guest",msg="Good Morning"): →Valid

def wish(name,msg="Good Morning"): →Valid

def wish(name="Guest",msg): →Invalid
```

SyntaxError: non-default argument follows default argument

Variable Length Arguments:

Sometimes we can pass variable number of arguments to our function,such type of arguments are called variable length arguments.

We can declare a variable length argument with * symbol as follows.

def f1(*n):

We can call this function by passing any number of arguments including zero number Internally all

In [39]:
```
def sum(*n):
    total=0
    for n1 in n:
        total=total+n1
    print("The Sum=",total)
sum(10)
sum(10,20)
sum(10,20,30,40)
```

```
The Sum= 10
The Sum= 30
The Sum= 100
```

We can mix variable length arguments with positional arguments.

In [41]:
```
def  f1(n1,*s):
    print(n1)
    print(s)
    for s1 in s:
        print(s1)
f1(10)
f1(10,20,30,40)
f1(10,"A",30,"B")
```

```
10
()
10
(20, 30, 40)
20
30
40
10
('A', 30, 'B')
A
30
B
```

After variable length argument, if we are taking any other

arguments then we should provide values as keyword arguments.

In [45]:
```
def  f1(*s,n1):
    for s1 in s:
        print(s1)
    print(n1)
f1("A","B",n1=10)
```

```
A
B
10
```

In [46]:
```
f1("A","B",10)  #Invalid
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-46-15d075b87953> in <module>
----> 1 f1("A","B",10)  #Invalid

TypeError: f1() missing 1 required keyword-only argument: 'n1'
```

We can declare key word variable length arguments also.

For this we have to use **.

def f1(**n):

We can call this function by passing any number of keyword arguments.

Internally these keyword arguments will be stored inside a dictionary.

In [51]:
```
def  display(**kwargs):
    for k,v in kwargs.items():
        print(k,"=",v)
display(m1=100,m2=200,m3=300)
```

```
m1 = 100
m2 = 200
m3 = 300
```

In [52]:
```
display(rno=100,name="Arman",marks=88,subject="Java")
```

```
rno = 100
name = Arman
marks = 88
subject = Java
```

Types Of Variables:

Python supports 2 types of variables.

Global Variables

Local Variables

Global Variables:

The variables which are declared outside of function are called global variables.

These variables can be accessed in all functions of that module.

In [53]:
```
a=10  # global variable
def f1():
    print(a)
def f2():
    print(a)
f1()
f2()
```

```
10
10
```

Local Variables:

The variables which are declared inside a function are called local variables.

Local variables are available only for the function in which we declared it.

i.e from outside of function we cannot access.

In [54]:
```
def f1():
    a=10
    print(a)  # valid
f1()
```

```
10
```

In [1]:
```
def f2():
    print(a)  # Invalid
f2()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-1-f09a8bd6686a> in <module>
      1 def f2():
      2     print(a)  # Invalid
----> 3 f2()

<ipython-input-1-f09a8bd6686a> in f2()
      1 def f2():
----> 2     print(a)  # Invalid
      3 f2()

NameError: name 'a' is not defined
```

The Global Keyword:

When you create a variable inside a function,that variable is local,and can only be used inside that function.

In [3]:
```
def myfunc():
    global a
    a= "fantastic"
myfunc()
print("Python is " + a)
```

```
Python is fantastic
```

In [5]:
```
#Use the global keyword if you want to change
#a global variable inside a function.
a = "awesome"
def myfunc():
    global a
    a = "fantastic"
myfunc()
print("Python is " + a)
```

```
Python is fantastic
```

In [6]:
```
a = "awesome"
def myfunc():
    #global a
    a = "fantastic"
myfunc()
print("Python is " + a)
```

```
Python is awesome
```

In [28]:
```
#Find out max number among 3 numbers using udf.
def max(a,b,c):
    if (a>b) and (a>c):
         print(a,"is max")
    elif(b>c):
         print(b,"is max")
    else:
        print(c,"is max")
a=int(input("Enter number 1: "))
b=int(input("Enter number 2: "))
c=int(input("Enter number 3: "))
max(a,b,c)
```

```
Enter number 1: 45
Enter number 2: 86
Enter number 3: 12
86 is max
```

In [7]:
```
#Write a python program to check given number is armstrong number
#or not with udf.
def armstrong(n):
    temp=int(n)
    sum=0
    a=len(n)
    while temp!=0:
        d=temp%10
        sum=sum+(d**a)
        temp=temp//10
    if(sum==int(n)):
        print("Number is armstrong.")
    else:
        print("Number is not armstrong.")
n=input("Enter number: ")
armstrong(n)
```

```
Enter number: 153
Number is armstrong.
```

In [8]:
```
#Local -global variable
x=3 # global
def fun(a):
    x=5 # local
    return x*a
print(fun(5))
print(x)
```

```
25
3
```

In [9]:
```
#find factorial of given number using udf
def fact(n):
    factorial=1
    if(n<0):
         print("Negative numbers are not allowed.")
    elif n==1 and n==0:
         print("factorial= ",factorial)
    else:
         for i in range(2,n+1):
                factorial=factorial*i
    print("Factorial of number",n,"is ",factorial)
n=int(input("Enter the number :"))
fact(n)
```

```
Enter the number :5
Factorial of number 5 is  120
```

Recursive Function

Recursive functions are functions that calls itself.

It is always made up of 2 portions, the base case and the recursive case.

The base case is the condition to stop the recursion.

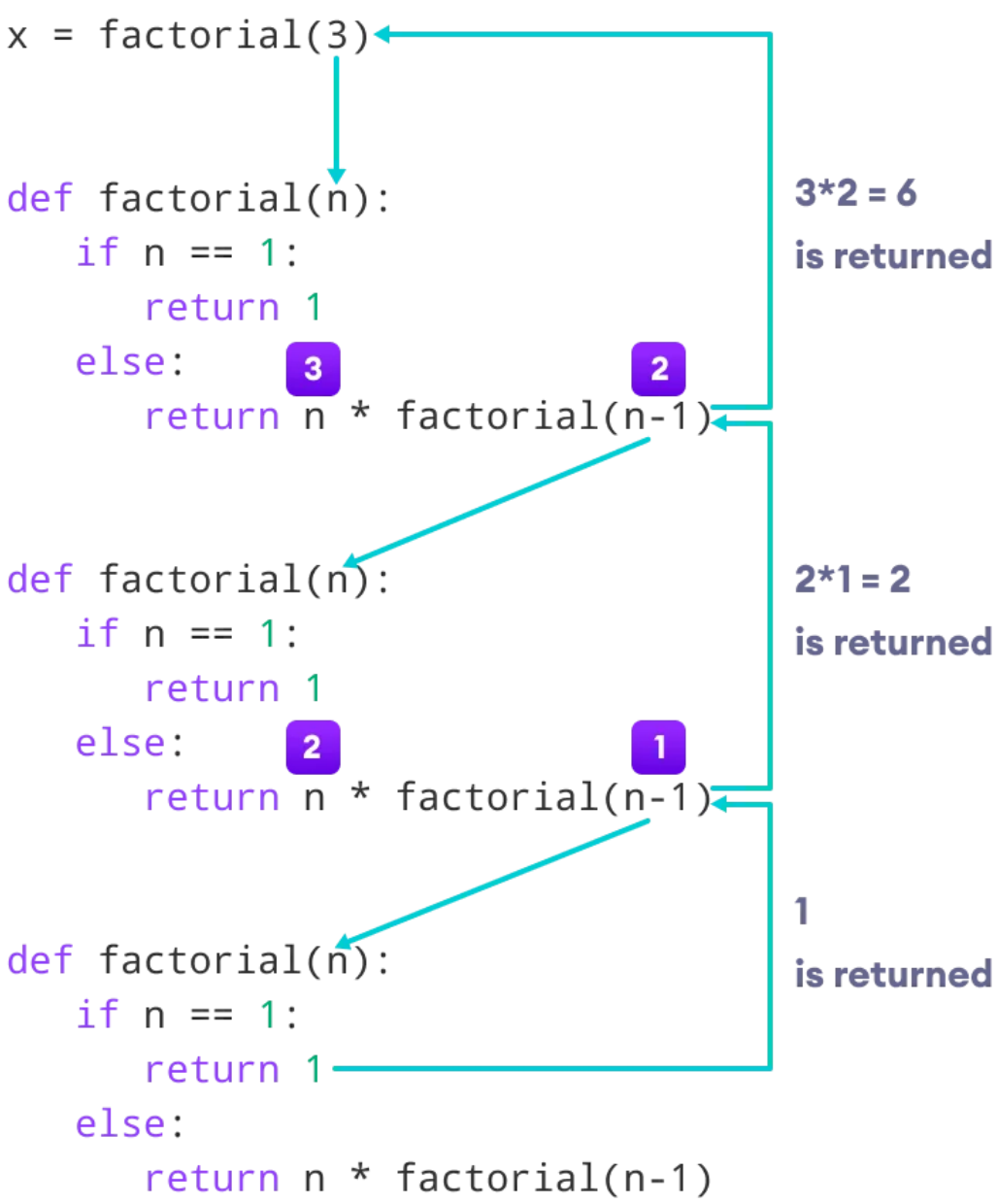
The recursive case is the part where the function calls on itself.

In [12]:
```
def factorial(x):
    """This is a recursive function
    to find the factorial of an integer"""

    if x == 1:
        return 1
    else:
        return (x * factorial(x-1))

num = 3
print("The factorial of", num, "is", factorial(num))
```

```
The factorial of 3 is 6
```



Nested function:

A nested function refers to a function that is defined within another function's body.

In programming languages that support this feature (like Python),

the inner function can access variables and parameters from its enclosing (outer) function,

even after the outer function has finished executing.

In [21]:
```
# creating an outer function
def outerFunc(sample_text):
   sample_text = sample_text
   print(sample_text)
# creating an inner function
   def innerFunc():
# Printing the variable of the parent class(outer class)
      print(sample_text)
# calling inner function
   innerFunc()
# Calling the outer Function by passing some random name
outerFunc('Hello  python')
```

```
Hello  python
Hello  python
```

In [23]:
```
# creating an outer function
def outerFunc(sample_text):
    sample_text = sample_text
    # creating an inner function
    def innerFunc():
       print(sample_text)
 # calling inner function
    innerFunc()

#NO OUTPUT, OUTER FUNCTION NOT CALLED
```

In [24]:
```
# creating an outer function
def outerFunc():
   sample_str = 'Hello'
# creating an inner function
   def innerFunc():
   # modifying the sample string
      sample_str = 'Welcome'
      print(sample_str)
# calling an inner function inside the outer function
   innerFunc()
   print(sample_str)
# calling outer function
outerFunc()
```

```
Welcome
Hello
```

In [2]:
```
def number_1(a):
   def number_2(b):
      print(a)
      print(b)
      return a*b
   return number_2
print(number_1(3)(2))
```

```
3
2
6
```

In [27]:
```
#How to pass Python function as a function argument?
def apply_function(func, x):
   return func(x)
def square(x):
   return x ** 2
result = apply_function(square, 3)
print(result)
```

```
9
```

pass by referance and pass by value

In [3]:
```
def set_val(val):  #pass by value
    val = "A"
    return val

def add(val): #pass by referance
    val += "A"
    return val

my_val = "E"

print(set_list(my_val))

print(add(my_val))
```

```
A
EA
```

In [ ]: